

Hoisting in JavaScript is the behavior where variable and function declarations are moved (“hoisted”) to the top of their scope before code execution. This means you can use functions and variables before they are actually declared in the code — but with some important differences between `var`, `let`, `const`, and function declarations.

Key Points

- Function Declarations** are fully hoisted. You can call them before they appear in the code.
- Variables declared with `var`** are hoisted but initialized with `undefined`.
- Variables declared with `let` and `const`** are hoisted but remain in a *temporal dead zone* until their declaration is reached. Accessing them before declaration throws an error.

Code Snippets

1. Function Hoisting

```
sayHello();  
  
function sayHello() { console.log("Hello, world!"); }
```

Here, the function declaration is hoisted, so calling it before definition works.

2. Variable Hoisting with `var`

```
````javascript  
console.log(a); // undefined
var a = 10;
console.log(a); // 10
```

The declaration `var a` is hoisted, but its assignment happens later. So the first log prints `undefined`.

---

### 3. Variable Hoisting with `let` and `const`

```
console.log(b); // ReferenceError
let b = 20;

console.log(c); // ReferenceError
const c = 30;
```

Both `let` and `const` are hoisted but not initialized. Accessing them before declaration causes a **ReferenceError**.

---

Analogy Think of hoisting like reserving seats in a theater:

- `var` reserves the seat but leaves it empty until later.
- `let` and `const` reserve the seat but don't let you sit until the right time.
- Functions not only reserve the seat but also bring the person in immediately.